CONFIDENTIAL · CONSULTANT QUOTATION BRIEF

Silver Carz
Rental & Fleet Management System

Project scope, architecture, and implementation summary for an independent industry expert to assess commercial quoting to the client.

Client / brand	Silver Carz, Nagpur, Maharashtra (India)
Product type	Custom dual-portal web application (admin operations + customer booking)
Build period	26 July 2026 – 12 August 2026 (active development; ~2.5 weeks)
Document date	14 August 2026
Purpose	Help a consultant estimate a fair client quote. This document does not recommend a price.

> **How to use this brief.** Treat this as a scope pack, not a sales deck. It describes what was actually built, how it was engineered, what is production-ready, and what is still incomplete. Quote against delivered capability plus remaining go-live work, not against a generic “car rental website.”

Prepared from the live codebase (TypeScript/Next.js application, Supabase schema, and internal docs). Figures are approximate and intended for commercial scoping, not a formal audit.

1. Executive summary

Silver Carz commissioned a **custom rental operations system**, not a brochure site. The product is a dashboard-first fleet and booking platform for internal staff, plus a public customer portal where people can browse cars, request a hire, upload KYC documents, wait for staff approval, and start an online payment.

The work is closer to a **small custom SaaS / operations product** than to a WordPress or template build. Business rules (availability, schedule conflicts, hire pricing, invoice numbers, booking lifecycle) live in dedicated server-side engines so they are not duplicated in the UI. Security is enforced in Postgres with Row Level Security, not only in the frontend.

What this is

- Two branded portals in one codebase
- Staff RMS: fleet, bookings, calendar, KPIs
- Customer site: browse → request → documents → pay
- India-specific money, KYC, and Razorpay checkout

What this is not (yet)

- Not a fully closed payment loop (verification still pending)
- Not a complete CRM (customers / drivers / settings are placeholders)
- Not car pooling, detailing, or marketing CMS
- Not invoice PDF printing or GST-ready invoicing

Quoting implication: the hard custom work (dual auth, RLS, scheduling engines, KYC storage, payment order creation) is largely done. Remaining work is concentrated in payment confirmation, a few operational modules, and go-live hardening — not in rebuilding the product.

2. Client context and product intent

Silver Carz is a vehicle rental business in Nagpur. The system is meant to replace informal (spreadsheet / WhatsApp / walk-in) operations with a single source of truth for:

- Which cars are free, reserved, on hire, in maintenance, or retired
- Who booked which vehicle, for which dates, at which rate
- Staff-created walk-in bookings and customer-originated requests in one queue
- Document collection before a car is released
- Hire money (daily rate × days, extra km, advance, remaining balance) in INR

Staff scale assumed in the original product brief is small (Owner + Manager roles; a handful of internal users). The customer portal is public and unbounded. Currency, locale, and document types are India-specific (INR, `en-IN`, driving licence / government ID / address proof, Razorpay).

3. What was implemented

3.1 Admin portal STAFF

Internal application at `/admin`. Email/password login. Only Owner and Manager (staff allowlist) can enter. Customers who log in here are rejected.

Module	What staff can do	Status
Authentication	Email/password login, cookie sessions, inactive-account lockout. Password reset is admin-handled (no self-service).	**Done**
Dashboard	Post-login operations home: KPI cards, booking-status pie, fleet-availability bars, today's pickups/returns, recent bookings, fleet snapshot, live clock.	**Done**
Bookings — walk-in	Full CRUD-style workspace: searchable/filterable list, create, edit, details. Shared form for create/edit. Soft-cancel (status = cancelled). Print invoice is a disabled placeholder.	**Done (print open)**
Bookings — customer queue	Separate "Pending approval" vs "Confirmed" tabs. Staff review uploaded KYC (signed preview URLs), then approve or deny with a written reason.	**Done**
Vehicles / fleet	List with search/filters/pagination; add/edit vehicle; details profile; image upload to Storage; availability and soft-retire; recent bookings on the vehicle record.	**Done**
Fleet calendar	Day / week / month occupancy timeline. Range-scoped data load. Click through to booking details. Read-only (no drag-and-drop scheduling).	**Done (no DnD)**
Customers / Drivers / Settings	Sidebar entries exist as placeholders. No CRM, driver roster, or settings UI.	**Not built**

3.2 Customer portal PUBLIC

Public site at `/` with a distinct yellow/black/white brand (same component library, different design tokens). One Supabase Auth project serves both portals; roles split the experience.

Module	What customers can do	Status
Book a Car	Browse active fleet, filter, select a vehicle, see a summary and daily rate. Public read is RLS-limited to active vehicles.	**Done**
Customer auth	Sign up and login. Public signup always creates `customer` role. Staff emails are allowlisted separately so a public signup cannot become Owner/Manager.	**Done**
Booking request wizard	Authenticated multi-step flow: dates (with booked-date calendar), trip details, review. Server re-checks availability and conflicts. Draft is saved locally between steps.	**Done**
KYC documents	Upload driving licence, government ID, and address proof (PDF/JPEG/PNG, max 5 MB each) into a *private* Storage bucket. Completeness gate before staff review.	**Done**
My bookings + request status	List own requests; see pending / rejected (with reason) / approved / payment-required states.	**Done**
Online payment	After approval, Pay Now creates a Razorpay order server-side, records a pending attempt, and opens Checkout. Webhook verifies signature and stores gateway payment id.	**Partial — see §7**
Car pooling / detailing / about / profile / legal	Routes exist as branded placeholders. No booking, CMS, or account-profile editing.	**Not built**

3.3 End-to-end hire journey (as implemented)

```
Customer browses fleet (public)
  → signs up / logs in (role = customer)
  → picks vehicle → date calendar (blocked days from Conflict Engine)
  → trip details + estimated total (Pricing Engine)
  → submit request → draft booking + invoice number allocated
  → upload 3 KYC files (private Storage)
  → staff reviews documents on admin booking detail
  → staff Approve  or  Deny + reason
  → if approved: customer Pay Now (Razorpay Checkout)
  → webhook stores provider payment id
  → confirmation screen shows “processing”
  → [NOT YET] authoritative mark-paid + booking collection update
```

4. How it was built (architecture)

The implementation is a production-style layered Next.js application, not a page-by-page prototype. That matters for quoting: the client is buying a maintainable product, not a one-off demo.

4.1 Application shape

- **Thin routes.** Files under `src/app/` only compose feature UI. Business logic is not in pages.
- **Feature modules.** Domain code lives in `src/features/`: auth, bookings, vehicles, calendar, dashboard, customer-auth, customer-booking, booking-documents, payments.
- **Layering per feature.** Repository (Supabase/SQL) → Service (rules + `ApiResponse`) → Server Action (UI entry) → React components.
- **Server Components by default.** Client components only where interactivity is required (forms, tables, charts, checkout).
- **One backend project.** Supabase Postgres + Auth + Storage. No separate custom API server.

4.2 Dual-portal split

Concern	Admin	Customer
URL space	`/admin/*`	`/`, `/login`, `/booking/…`
Roles	owner, manager (staff allowlist)	customer (default on public signup)
Visual identity	Gold-on-light/dark operations UI, sidebar shell	Yellow / black / white marketing + booking chrome
Auth screens	`/admin/login`	`/login`, `/signup`
Data access	Full fleet and all bookings via staff RLS	Own bookings, public active vehicles, own documents/payments

Theme switching is configuration-driven (CSS variables + `data-portal`), so both portals share shadcn/ui primitives without forking the component library. A vendor theme preset exists for a possible third portal later; it is not implemented as a product.

4.3 Centralized domain engines (high-value custom logic)

These are the pieces that typically justify custom-software pricing. UI never invents the math or overlap rules; every surface calls the same services.

Engine	Responsibility	Why it is non-trivial
Vehicle Availability	available / reserved / booked / maintenance / inactive; sync from bookings; staff override	Booking-driven state plus manual maintenance that must not be overwritten
Conflict Detection	Inclusive date-window overlap per vehicle; blocking vs ignored statuses	Same-day handoff conflicts; adjacent days do not; draft/cancelled ignored; edit excludes self
Status Automation	upcoming / active / completed from dates; cancelled/denied/draft override	Display status vs persisted DB status; used by calendar, dashboard, availability
Pricing	Inclusive rental days, km charge, subtotal, grand total, remaining balance	Single source of truth; GST/discount hooks exist but rates are 0 today; security deposit is not netted off balance
Invoice numbering	Atomic `SC-YYYY-XXXXX` via Postgres RPC (yearly sequence)	Concurrency-safe; client cannot supply or rewrite numbers

4.4 Data and security model

- **Postgres tables:** `profiles`, `vehicles`, `bookings`, `invoice_sequences`, `booking_documents`, `payments`, `staff_allowlist`.
- **RLS enabled (and forced) on business tables.** Anonymous users cannot read staff data. Customers can only see their own bookings/documents/payments. Staff helpers are `SECURITY DEFINER` role checks — not JWT `user_metadata`.
- **Storage:** public bucket for vehicle images; private bucket for KYC. Previews use short-lived signed URLs (staff ~2 minutes). Path-scoped policies; customers cannot list other people's files.
- **Service role** is server-only (webhooks). Never shipped to the browser.
- **Validation:** Zod schemas shared by forms and Server Actions. Client input is not trusted for rates, invoice numbers, status, or `created_by`.

5. Technology stack

Layer	Choice	Notes for quoting
Framework	Next.js 16 (App Router), React 19	SSR/RSC, Server Actions, Proxy for session refresh
Language	TypeScript (strict; `any` disallowed)	Higher build quality / slightly higher rate than typical JS freelance work
Styling / UI	Tailwind CSS v4, shadcn/ui, Radix, Lucide, Framer Motion	Custom design system, not a bought theme
Forms / tables / charts	React Hook Form, Zod, TanStack Table, Recharts	Standard professional stack
Server state	TanStack Query	Client cache where needed
Backend	Supabase (PostgreSQL, Auth, Storage, RPCs)	Managed BaaS; client still owns schema, RLS, and migrations

Layer	Choice	Notes for quoting
Payments	Razorpay (Orders + Checkout + webhook)	India card/UPI/netbanking; PCI burden on Razorpay
Hosting target	Vercel + hosted Supabase	Typical monthly infra is low relative to build cost
Tooling	pnpm, ESLint, Prettier, Husky, lint-staged, Conventional Commits	Pre-commit typecheck + lint — production hygiene

Locale/currency: INR, `en-IN`, `dd MMM yyyy`. Dates via date-fns. Package manager pnpm 10; Node 20+.

6. Engineering quality (relevant to price)

Practices in place

- ~70 Conventional Commits on the main line
- ~24 SQL migrations with comments and apply order
- ~24 internal architecture docs (engines, RLS, UI)
- Normalized errors (no raw DB messages in UI)
- Loading / empty / error / not-found states on key screens
- Responsive audit of the admin portal

Approximate size

- ~391 TypeScript/TSX source files
- ~35,700 lines in `src/`
- ~2,700 lines of SQL migrations
- ~3,800 lines of product documentation
- 9 feature modules
- 26 routed pages (plus API webhook)

Size alone does not set price, but it indicates this is not a 5-page marketing site. A comparable custom dual-portal operations product is typically scoped as a **mid-complexity custom web application** with a payment and KYC increment.

7. Payments — what is done vs what is not

This is the most important commercial caveat. Online payment is **integrated but not closed**.

Capability	Status	Detail
Eligibility gate	**Done**	Pay only after approval, documents submitted, remaining balance > 0, not cancelled/denied
Razorpay order creation	**Done**	Server-side; amount from Pricing Engine remaining balance; receipt tied to invoice number
Pending payment row	**Done**	`payments` table with RLS; customers cannot self-mark paid
Checkout UI + retry/cancel	**Done**	Browser Checkout; failed/cancelled attempts are recorded
Webhook signature verify	**Done**	`POST /api/payments/razorpay/webhook`; attaches `provider_payment_id`
Authoritative mark-paid (C7)	**Not done**	Does *not* set payment = paid, does *not* write `booking_amount`, does *not* treat Checkout success as proof

Staff can still record cash/UPI/card on walk-in bookings via the admin form (`payment_method` + `booking_amount`). That path is independent of Razorpay.

8. Explicitly not included (open / later phases)

Item	Impact if the client expects it in the quote
Payment verification & confirmation (C7)	Must be finished before taking live customer money with confidence
GST, discounts, included-km packages, invoice PDF / print	Pricing engine has hooks; no tax invoices or print output yet
Customer CRM, driver roster, admin user-management UI	Nav placeholders only
Customer profile editing, self-service password reset	Not implemented
Car pooling, car detailing, About Us, legal pages	Branded empty routes
Calendar year view, drag-and-drop reschedule	Calendar is view/navigate only
Notifications (email/SMS/WhatsApp)	None
Reports / accounting export / multi-branch	None
Native mobile apps	Responsive web only
Per-file KYC approve/reject workflow	Staff can view files; approval is at booking level
Vendor portal	Theme tokens only

9. Suggested quoting frame (for the consultant)

The author of this brief is the developer, not a pricing authority. The following is a **work-breakdown the consultant can price**, not a recommended rupee figure.

9.1 Work already delivered (should be valued)

1. Product architecture, design system, and dual-portal shell
2. Auth, RBAC, staff allowlist, RLS, and session/proxy hardening
3. Fleet CRUD + images + availability engine
4. Admin booking workspace + customer request/approval workflow
5. Conflict, status, pricing, and invoice-number engines
6. Operations dashboard and occupancy calendar
7. Private KYC document pipeline
8. Razorpay checkout infrastructure (order + pending + webhook attach)

9.2 Remaining to a responsible go-live

1. **Must-have:** payment verification (C7), webhook idempotency tests, live Razorpay keys, failure/refund policy
2. **Must-have:** production env (Vercel + Supabase prod), backups, staff training, seed/allowlist
3. **Should-have:** invoice PDF, basic email/SMS on request/approval/payment

4. **Should-have:** customer profile, password reset, terms/privacy pages
5. **Nice-to-have:** customers/drivers modules, GST, reports, pooling/detailing

9.3 Complexity signals (use these, not page-count)

- **Two products in one:** internal RMS + public booking site, shared data, split auth.
- **Domain rules:** scheduling + money + KYC + payments is a higher band than CRUD.
- **Security surface:** PII documents in private storage; payment webhooks; role isolation.
- **India payments:** Razorpay, INR, UPI-capable checkout; remaining verification work is specialized, not cosmetic.
- **Code quality:** typed, documented, migrated, lint-gated — maintenance cost should be lower than a throwaway build, so a warranty/retainer is reasonable.
- **Calendar time:** ~2.5 weeks of dense delivery is a velocity signal, not a ceiling on commercial value. Custom dual-portal systems of this shape are often quoted as a multi-week / multi-month product even when a fast developer compresses the calendar.

9.4 Commercial models the consultant may consider

Model	When it fits
Fixed fee for delivered MVP + punch-list to go-live	Client wants one number for "what exists + make payments live"
Phased: (A) current build, (B) go-live/C7, (C) CRM/invoices/GST	Client may not need pooling/detailing now; keep later work optional
Build fee + monthly retainer	Hosting, Razorpay issues, small change requests, backups, staff support

Recurring costs the quote should mention separately (usually pass-through, not developer margin): Vercel, Supabase, Razorpay MDR, SMS/email provider, domain/DNS.

Do not quote this as a "website." Quote it as a custom operations + customer booking product with KYC and payments. If the client compares it to a ₹20–40k brochure site, the comparison is the wrong category. If they compare it to an off-the-shelf rental SaaS subscription, the trade-off is ownership and Silver Carz-specific workflow vs monthly SaaS fees.

10. Risks and assumptions a consultant should flag

- **Live money:** do not enable live Razorpay until C7 is done and webhook/replay cases are tested.
- **KYC/PII:** driving licence and Aadhaar-class files are in private storage; retention/deletion policy is not productized.
- **Owner vs Manager:** permission matrix exists but both roles currently have full access.
- **Customer identity** is still denormalized on the booking row (name/phone/address), not a full customer master.
- **No automated test suite** of note; quality is architectural + manual. A go-live quote may include smoke tests around payments and RLS.
- **README in the repo is stale** (it still describes an admin-only MVP). Trust this brief and the code over that README.

11. One-page snapshot

Question	Answer
What did the developer build?	A custom dual-portal fleet rental system for Silver Carz: staff RMS + customer booking/KYC/payment start.
How?	Next.js 16 + TypeScript + Supabase, feature modules, Server Actions, RLS, and five centralized domain engines.
Is it a finished commercial product?	Core operations and request workflow: yes. Authoritative online payment confirmation and several sidebar modules: no.
Main remaining paid work?	C7 payment verification, go-live ops, invoice PDF / notifications, then optional CRM and GST.
Category for pricing?	Mid-complexity custom web application (operations SaaS-like), India payments + KYC, not a marketing website.

End of brief. For technical deep-dives the repository contains `docs/` (architecture, authentication, database, pricing, conflict, availability, calendar, dashboard). This document is for commercial scoping only and is not a legal statement of work.